

What is a Stack Data Structure?

A stack is a linear data structure that follows the principle:

LIFO – Last In, First Out: The **last element inserted** into the stack is the **first one to be removed**.

Applications of Stack

Application	Explanation
Function Calls	Call stack
Expression Evaluation	Infix → Postfix
Parenthesis Checking	((a+b))
Undo/Redo	Editors
Backtracking	Maze, DFS
Reversal	String/Array reversal

Basic Terminology in Stack

Term	Meaning
Top	keeps track of the index of the last inserted element in the stack.
Push	Insert element into stack
Pop	Removes & returns top element
Overflow	Stack is full
Underflow	Stack is empty

Stack Operations (Methods): These are the core methods of a stack:

- 1 Push (Insertion):** Adds an element to the **top** of the stack.
- 2 Pop (Deletion):** Removes & returns the **top element** from the stack.
- 3 Peek / Top:** Returns the **top element** without removing it.

- 4 **isEmpty()**: Checks whether the stack is empty.
- 5 **isFull()**: Checks whether the stack is full.
- 6 **Size()**: Returns the number of elements in the stack.

Stack Representation

A stack can be implemented using:

1.Array

- **Declare an Array of Particular Size**: Initialize an array that will hold the elements of the stack. The size of the array is defined when the stack is created.
- **Define a Variable "Top" and Initialize It as -1**: The "top" variable keeps track of the index of the last added element in the stack. Initializing it to -1 indicates that the stack is empty.
- **push(int x)**: To push an element onto the stack:
 - If the stack is full (top is equal to the last index of the array), throw a stack overflow exception.
 - Increment the top index by one and insert the element at this position in the array.
- **pop()**: To pop an element from the stack:
 - If the stack is empty (top == -1), throw a stack underflow exception.
 - If the stack is not empty, return the element at the top index and then decrement the top index by one.
- **top()**: To get the top element without removing it:
 - Check if the stack is not empty. If it is empty, throw an exception.
 - If the stack is not empty, return the element at the top index.
- **size()**: To get the current size of the stack:
 - Return top + 1.

Time Complexity: $O(1)$ for all operations (push, pop, top, isEmpty).

Space Complexity: $O(N)$, where N is the maximum capacity of the stack, as we are using an array to store the elements.

2. Linked List

- **Node Structure:**
 - Define a node with:
 - An integer to store data.
 - A pointer to the next node.
 - A constructor to initialize the data and the next pointer.
- **Stack Structure:**
 - Define a stack with:
 - A pointer to the top node.
 - An integer to keep track of the size.
 - A constructor to initialize the top pointer and size.
- **Push Operation:**
 - Create a new node with the given data and set the new node's next pointer to the current top node.
 - Update the top pointer to the new node and increment the size.
- **Pop Operation:**
 - If the stack is empty(**top==null**), return an error value (e.g., -1).
 - Store the data of the top node and update the top pointer to the next node.
 - Decrement the size. Return the stored data.
- **Peek Operation:**
 - If the stack is empty(**top==null**), return an error value (e.g., -1).
 - Otherwise, return the data of the top node.
- **Size Operation:**
 - Return the size of the stack.
- **Print Stack:**
 - Traverse from the top node and print each node's data until the top becomes null.

3. Using single Queue

- **Data Structure Used:** A single queue will be used to store the elements.
- **push(x):** Insert the element x into the queue. To maintain the stack order:
 - Run a loop that iterates $\text{size}() - 1$ times, where $\text{size}()$ is the current number of elements in the queue.
 - In each iteration, remove the front element and add it back to the rear of the queue. This ensures that the most recently added element is always at the front of the queue.
- **pop():** Remove and return the front element of the queue, which corresponds to the top of the stack.
- **isEmpty():** Return true if the queue is empty, and false otherwise.

Time Complexity:

Push operation: $O(n)$ (where n is the number of elements in the queue at that time) because every time an element is pushed, all the elements in the queue are popped from the front and pushed in the back again.

Pop operation: $O(1)$ as constant operations are performed.

Top operation: $O(1)$ as constant operations are performed.

IsEmpty operation: $O(1)$ as constant operations are performed.

Space Complexity: $O(k)$ for storing k elements in the queue.

What is a Queue Data Structure?

A **Queue** is a **linear data structure** that follows the principle:

FIFO – First In, First Out

means: The **first element inserted** is the **first one to be removed**

Basic Terminology in Queue

Term	Meaning
Front	Points to the first inserted element
Rear	Points to the last inserted element
Enqueue	Insert element

Term	Meaning
Dequeue	Remove element
Overflow	Queue is full
Underflow	Queue is empty

Core Queue Operations (Methods)

- 1 Enqueue (Insertion):** Adds an element at the **rear** of the queue.
- 2 Dequeue (Deletion) / poll():** Removes & returns an element from the front of the queue.
- 3 Peek / Front:** Returns the front element without removing it.
- 4 isEmpty():** Checks whether the queue is empty.
- 5 isFull():** Checks whether the queue is full.
- 6 Size():** Returns the number of elements in the queue.

Queue Representation

Queue can be implemented using:

1.Array (linear queue)

- **Declare an Array of Particular Size:**
- **Define Two Variables “Front” and “Rear” and Initialize Them:**
 - The front variable keeps track of the first element of the queue, and the rear variable keeps track of the last element.

Initially: front = 0 & rear = -1

This indicates that the queue is empty.

-
- **enqueue(int x):** To insert an element into the queue:
 - If the queue is full (rear is equal to the last index of the array), throw a **queue overflow** exception.
 - Increment the rear index by one & Insert the element at the rear position in the array.

-
- **dequeue() / poll():** To remove & return an element from the queue:
 - If the queue is empty ($\text{front} > \text{rear}$), throw a **queue underflow** exception.
 - If the queue is not empty, return the element at the front index & Increment the front index by one.
-

- **peek() / front():** To get the front element without removing it:
 - If queue is empty, throw an exception.
 - If the queue is not empty, return the element at the front index.
-

- **size():** To get the current size of the queue:
 - Return $(\text{rear} - \text{front} + 1)$.
-

Important Note (Normal Queue Limitation)

In a normal array-based queue:

- Dequeued spaces at the front **cannot be reused**
- This leads to **wasted space**
- This drawback is solved using a **Circular Queue**.

2. Circular Array

- Declare an Array of Particular Size:
- Define Variables front, rear, size(represent total size of queue), and currSize(represents total number of elements in the queue)

Initially: $\text{front} = -1$, $\text{rear} = -1$, $\text{currSize} = 0$

This represents an empty queue.

- **add(int element)**
 - If $\text{currSize} == \text{size}$, queue is full return.

- Else If the queue is empty (both front and rear are -1), set both front and rear to 0.
 - Else, increment the rear index using modulo operation to maintain circular behavior.
 - Insert the element at the rear position.
 - Increment the current size of the queue.
-

- **poll():**

- If the current size of the queue is 0, a **queue underflow** condition occurs.
 - Store the element present at the front index.
 - If the queue contains only one element (front == rear), reset both front and rear to -1.
 - Else, increment the front index using modulo operation to move circularly.
 - Decrement the current size of the queue.
 - Return the removed element.
-

- **peek():**

- To get the front element without removing it:
 - If the queue is empty, return an error value.
 - Otherwise, return the element present at the front index.
-

- **size():**

- Return the value of currSize.

Key Advantages of This Implementation

- Efficient utilization of array space using circular indexing
- No memory wastage as seen in normal (linear) queue
- All operations run in **O(1) time**

3. Linked List

- **Node Structure:** Define a node that holds data and a pointer to the next node. This node acts as the building block of the linked list used in the queue.

- **Queue Initialization:** Initialize the queue with pointers to both the front and rear of the queue. Set these pointers to null initially, indicating an empty queue. Also, maintain a size to track the number of elements in the queue.
- **Enqueue Operation (Adding an Element):**
 - Create a new node with the given data.
 - If the queue is empty($\text{front} == \text{null}$), set both the front and rear pointers to this new node.
 - Else, link the rear node to the new node and update the rear pointer to point to the new node & increase size by 1.
- **Dequeue Operation (Removing an Element):**
 - If the queue is empty return -1;
 - Store reference of front into 'temp'.
 - If list only contains only a single node, then set both rear & front to null.
 - Else move front by one node.
 - Decrease size by 1 & return value stored inside the 'temp' node.
- **Peek Operation (Accessing the Front Element):**
 - If the queue is empty return -1;
 - Else, return the data of the front node without removing it.
- **Size Operation:** Return the value of the size tracking the number of elements in the queue.

4. Using two Stacks

Approach-1: Here, push operation will take $O(N)$

- **Use Two Stacks:** Maintain two stacks, stack1 and stack2.
- **Push Operation:**
 - Transfer all elements from stack1 to stack2.
 - Add the new element to stack1.
 - Transfer all elements back from stack2 to stack1.
 - This ensures the new element is always at the front for the next pop operation.

- **Pop Operation:** Remove and return the top element from stack1.
- **Top Operation:** Return the top element of stack1 without removing it.
- **Size Operation:** Return the size of stack1.

Time Complexity: $O(n)$ for push operation, $O(1)$ for pop and peek operations.

Space Complexity: $O(n)$ for storing elements in the stacks.

Approach-2: Here, push operation will take $O(1)$

- **Use Two Stacks:** Maintain two stacks, inputStack and outputStack.
- **Push Operation:**
 - Add the element to inputStack. This operation is efficient and always takes $O(1)$ time.
- **Pop Operation:**
 - If the outputStack is empty, move all elements from inputStack to outputStack. This reversal of order ensures that the oldest element is on top of the outputStack.
 - Remove and return the top element from outputStack. This represents the oldest element in the queue.
- **Top Operation:**
 - If the outputStack is empty, move all elements from inputStack to outputStack to access the oldest element.
 - Return the top element of outputStack without removing it. This gives the element that has been in the queue the longest.
- **Size Operation:**
 - Return the sum of the sizes of both stacks. This total gives the number of elements currently in the queue.

Time Complexity: $O(1)$ for push operation, $O(n)$ for pop and peek operations in the worst case when elements need to be shifted.

Space Complexity: $O(n)$ for storing elements in the two stacks.

Given a string *s* containing just the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Input: `str = "( ) [ { } ( ) ]"`

Output: True

Explanation: As every open bracket has its corresponding close bracket. Match parentheses are in correct order hence they are balanced.

Intuition

- We have to keep track of previous as well as most recent opening brackets and also keep in mind the sequence, as after opening of the bracket there should be opposite pairs of brackets.
- Also handle the corner cases like `[ ) ( ]` where closing bracket occurs first and opening bracket after it, which is an invalid sequence, as well as `[ ( ] )` where the most recent opening didn't get its opposite pair hence it will also not be valid.
- So we have to use a data structure that will keep track of first in and last out, hence we will use the stack.

Approach

- Whenever we get the opening bracket we will push it into the stack. I.e '(', '[', '{'.
- Whenever we get the closing bracket we will check if the stack is non-empty or not.
- If the stack is empty we will return false, else if it is nonempty then we will check if the topmost element of the stack is the opposite pair of the closing bracket or not.
- If it is not the opposite pair of the closing bracket then return false, else move ahead.
- After we move out of the string the stack has to be empty if it is non-empty then return it as invalid else it is a valid string.

Time Complexity: $O(n)$. Single for loop used

Space Complexity: $O(N)$. Stack space

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- **MinStack()** initializes the stack object.
- **void push(int val)** pushes the element val onto the stack.
- **void pop()** removes the element on the top of the stack.
- **int top()** gets the top element of the stack.
- **int getMin()** retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Approach - 1:

To solve this problem, we will use two (min, st) stack one is for storing element provided by the user & another is for keeping track of minimum element at each step. And whenever will perform the push() operation at that will push the element into the 'st' & min(element, min.peek()) in the 'min' stack. And whenever will perform the pop() at that time will remove one element from both stacks. To getMin() will simply return the top-most element of the 'min' stack.

Time: $O(1)$

Space: $O(2n)$ as we are using multiple stacks.

Approach - 2:

In this we will use a **stack(st)** & an integer **variable(min_val)** to keep track of min value.

- During push() if we are getting **value < min_val** will store modified value that is **(2*value – min_val)** in the stack & replace min_val with value. Otherwise, will simply store the value provided by the user.
- During pop(), will remove the top-most element from the stack & if **st.peek() < min_val** means stack's top has been modified & will replace the min_val with **(2*min_val – st.peek())** in-order to reach the previous min_value.

- During `top()` if `st.peek() < min_val` (means top has been modified & it is not the part of user input, will return the `min_val`). Else, will return the `st.peek()` (`st.peek() > min_val` & `min_val` has been modified).
- During `getMin()` will simply return `min_val`.

Time: $O(1)$, Space: $O(n)$ where n is the number of elements in the stack

Prefix, Infix & Postfix

Prefix: In a prefix expression, the operator appears before the operands.

`+ a b`

`* + a b c`

What is an infix expression?

- The traditional way of writing mathematical expressions is called infix expressions, where the operator is placed between two operands (e.g., $A + B$, $(A * B) / Q$).
- Infix expressions are easy for humans to understand, but computers find them difficult to parse because they require knowledge of operator precedence, associativity rules, and parentheses.
- To make it easier for computers, we use postfix and prefix notations.

What is a postfix expression?

- A postfix expression has the operator placed after the operands (e.g., $PQ-C/$). It is written as .
- In postfix expressions, the precedence of operators is determined by the order in which they appear in the expression. The operator that appears first is applied to the operands.
- Postfix expressions do not require parentheses, making them easier for computers to evaluate.

Infix to postfix

Input: $s = "a*(b+c)/d"$

Output: $abc+*d/$

Explanation: The expression is $a*(b+c)/d$. First, inside the brackets, $b+c$ becomes $bc+$. Now the expression looks like $a*(bc+)/d$. Next, multiply a with $(bc+)$, so it becomes $abc+*$. Finally, divide this result by d , so it becomes $abc+*d/$.

Approach:

- Use a stack to store operators and parentheses.
- Traverse the infix expression character by character.
- If operand (letter/digit) $\rightarrow$ directly append to postfix string.
- If '(' $\rightarrow$ push it onto the stack.
- If ')' $\rightarrow$ pop operators from stack and append to postfix until '(' is found, then remove '('.
- If operator (+ - * / ^):
 - Pop from stack while:
 - stack top has higher precedence, or
 - same precedence and operator is left-associative (^ is right-associative).
 - Push current operator onto stack.
- After traversal, pop all remaining operators from stack and append to postfix.
- Return the postfix expression.
- Finally, pop and print all remaining operators in the stack until it is empty.

Time Complexity: $O(N)$, where N is the length of the infix expression. Each character in the expression is processed once.

Space Complexity: $O(N)$, where N is the length of the infix expression. The stack can hold at most N operators and parentheses in the worst case.

Infix To Prefix

Input: $s = "a*(b+c)/d"$

Output: $/*a+bcd$

Explanation: The infix expression is $a*(b+c)/d$. First, inside the brackets, $b + c$ becomes $+bc$. Now the expression looks like $a*(+bc)/d$. Next, multiply a with $(+bc)$, so it becomes $*a+bc$. Finally, divide this result by d , so it becomes $/*a+bcd$.

Intuition:

- Reverse the infix expression along with that replace ')' with '(' & '(' with ')'.
• Convert reversed infix it to postfix. During postfix conversion take associativity of '^' from left-to-right & associativity of other operators from right to left.
• Reversing the postfix & return it as a result.

Time Complexity: $O(N)$, where N is the length of the infix expression. Each character in the expression is processed once.

Space Complexity: $O(N)$, where N is the length of the infix expression. The stack can hold at most N operators and parentheses in the worst case.

Postfix to Infix:

Input:

$ab*c+$

Output:

$((a*b)+c)$

Explanation:

The above output is its valid infix form.

Approach:

Here, We will traverse the given string from left to right & whenever will get any operator will simply push into the stack (type of string) otherwise, will remove two operands from the stacks & push the result of ' $(+ \text{second} + \text{operator} + \text{first} +)$ ' in the stack. Here first=first operand popped out from the stack & second=operand popped out second from the stack. Once the iteration gets completed will simply pop out the top-most element which will be the final result.

Time Complexity: $O(n)$, as we traverse the expression only once.

Space Complexity: $O(n)$ for the stack to store operands and intermediate results.

Postfix to Prefix:

Input:

ABC/-AK/L-*

Output:

*-A/BC-/AKL

Explanation:

The above output is its valid prefix form.

Approach:

Here, We will traverse the given string from left to right & whenever will get any operator will simply push into the stack (type of string) otherwise, will remove two operands from the stacks & push the result of **operator + second + first** in the stack. Here first=first operand popped out from the stack & second=operand popped out second from the stack. Once the iteration gets completed will simply pop out the top-most element which will be the final result.

Time Complexity: $O(n)$, as we traverse the expression only once.

Space Complexity: $O(n)$ for the stack to store operands and intermediate results.

Prefix to Infix:**Input:**

*-A/BC-/AKL

Output:

((A-(B/C))*((A/K)-L))

Explanation:

The above output is its valid infix form.

Approach:

Here, We will traverse the given string from right to left & whenever will get any operator will simply push into the stack (type of string) otherwise, will remove two operands from the stacks & push the result of '**(+ first + operator + second +)**' in the stack. Here first=first operand popped out from the stack & second=operand popped out second from the stack. Once the iteration gets completed will simply pop out the top-most element which will be the final result.

Time Complexity: $O(n)$, as we traverse the expression only once.

Space Complexity: $O(n)$ for the stack to store operands and intermediate results.

Prefix to Postfix:

Input:

*-A/BC-/AKL

Output:

ABC/-AK/L-*

Explanation:

The above output is its valid postfix form.

Approach:

Here, We will traverse the given string from right to left & whenever will get any operator will simply push into the stack (type of string) otherwise, will remove two operands from the stacks & push the result of **first + second + operator** in the stack. Here first=first operand popped out from the stack & second=operand popped out second from the stack. Once the iteration gets completed will simply pop out the top-most element which will be the final result.

Time Complexity: $O(n)$, as we traversed the expression only once.

Space Complexity: $O(n)$ for the stack to store operands and intermediate results.

Monotonic-Stack & Queue Problem

You are given an array `arr[]` of integers, the task is to find the next greater element for each element of the array in order of their appearance in the array. Next greater element of an element in the array is the nearest element on the right which is greater than the current element.

If there does not exist next greater of current element, then next greater element for current element is -1.

Input: `arr[] = [1, 3, 2, 4]`

Output: `[3, 4, 4, -1]`

Explanation: The next larger element to 1 is 3, 3 is 4, 2 is 4 and for 4, since it doesn't exist, it is -1.

Approach:

- Initialize an empty stack and a result array of the same length as input.
- Traverse the array from the last element to the first (right to left).
- For each element, pop elements from the stack while the stack top is less than or equal to the current element.
- If the stack becomes empty, no greater element exists, assign -1 in the result.
- Otherwise, the top of the stack is the next greater element for the current element.
- Push the current element onto the stack for use in future comparisons.

Time Complexity: $O(N)$, we traverse the entire array once and find next greater element in linear time.

Space Complexity: $O(2N)$, additional space used for resultant array and stack.

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return *the next greater number for every element in `nums`.*

The next greater number of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Input: `nums = [1,2,1]`

Output: `[2,-1,2]`

Explanation: The first 1's next greater number is 2;

The number 2 can't find next greater number.

The second 1's next greater number needs to search circularly, which is also 2.

Brute Force

- Create an array to store the answer of the same size as the input array, filled with -1.
- Iterate through each element at index `i` from 0 to `n-1`.
- For each element, start a second loop from 1 to `n-1` to check the next elements in circular order using $(i + j) \% n$.
- If a greater element than the current one is found during the inner loop, assign it to `answer[i]` and exit the inner loop.

- After completing both loops, return the answer array containing the next greater elements.

Time Complexity: $O(N^2)$, since we are using two nested for loops to find the next greater elements.

Space Complexity: $O(2N)$, additional space used for resultant array and stack.

Optimal Approach

- Initialize an answer array.
- Initialize an empty stack to keep track of elements
- Traverse from $2n - 1$ down to 0 using modulus to simulate circular indexing
- While stack is not empty and top of stack is less than or equal to current element, pop from stack
- If index is in the original array range, assign top of stack to answer if stack is not empty, else keep -1
- Push the current element onto the stack
- After traversal ends, return the answer array

Time Complexity: $O(4N)$, since traversing on the array takes $O(2N)$ time and traversing the stack will take overall $O(2N)$ time as all the elements are pushed in the stack once.

Space Complexity: $O(2N)$, since the answer array takes $O(N)$ space and the space used by stack will be $O(N)$ in the worst case.

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Approach-1:

- For each bar in the array, we pre-compute the maximum height bar on its **left(prefixSum) & right(suffixSum)** both.
- Calculate the trapped water on the current bar as $\min(\text{maxLeft}, \text{maxRight}) - \text{current height}$. If & only if height bar of both left & right are greater than the current bar.
- Sum all trapped water from each bar to get the total amount of trapped water.

```
static int bruteForce(int[] arr){ 1 usage
    int n = arr.length;

    int[] prefixMax = new int[n];
    prefixMax[0] = arr[0];
    for(int i=1;i<n;i++){
        prefixMax[i] = Math.max(prefixMax[i-1],arr[i]);
    }

    int[] suffixMax = new int[n];
    suffixMax[n-1] = arr[n-1];
    for(int i=n-2;i>=0;i--){
        suffixMax[i] = Math.max(suffixMax[i+1],arr[i]);
    }

    int totalWater = 0;
    for(int i=1;i<n-1;i++){
        if(arr[i]<prefixMax[i] && arr[i]<suffixMax[i]){
            totalWater += Math.min(prefixMax[i],suffixMax[i])-arr[i];
        }
    }
    return totalWater;
}
```

Time: $O(2n) + O(n)$

Space: $O(2n)$

Approach-2:

The optimal approach uses two pointers to efficiently calculate the trapped water without extra space for arrays. The key insight is that the amount of water trapped at any position depends on the tallest bars to the left and right of that position. By maintaining two pointers starting from both ends of the elevation array and tracking the maximum heights encountered so far from both sides, we can determine the trapped water at each step. If the left height is smaller or equal to the right height, the trapped water depends on the left side's maximum; otherwise, it depends on the right side's maximum. This two-pointer technique lets us compute trapped water in one pass while using constant extra space.

```
static int optimal(int[] arr){ 1 usage
    int left = 0, right=arr.length-1;
    int leftMax=-1, rightMax=-1;
    int totalWater = 0;
    while (left<right){
        if (arr[left]<arr[right]){
            if(leftMax>arr[left]) totalWater += leftMax-arr[left];
            leftMax = Math.max(leftMax,arr[left]);
            ++left;
        }
        else{
            if (rightMax>arr[right]) totalWater += rightMax-arr[right];
            rightMax = Math.max(rightMax,arr[right]);
            --right;
        }
    }
    return totalWater;
}
```

Time Complexity: $O(n)$ because the two pointers traverse the array only once, each pointer moving inward and covering the entire array in total linear time.

Space Complexity: $O(1)$

Given an array of integers arr, find the sum of $\min(b)$, where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo $10^9 + 7$.

Input: arr = [3,1,2,4]

Output: 17

Explanation:

Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].

Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.

Sum is 17.

Brute-force:

In this approach will run a outer loop for each array-element & an inner-loop for generating all the sub-arrays for each element in the outer loop. We will also use a totalSum variable to count total sum & an 'min' variable for keeping track of min-element inside the inner loop. And every time will update the 'min' & add 'min' to the sum. At the end will return totalSum.

Time Complexity: $O(N^2)$, since we are using two nested loops.

Space Complexity: $O(1)$,

Optimal:

Every subarray has **exactly one minimum**.

Instead of finding the minimum for each subarray (brute force), we **count how many subarrays each element is the minimum of**.

🔑 Key Idea

For an element arr[i]:

- Find how far it can extend to the **left** until a **smaller or equal** element appears
- Find how far it can extend to the **right** until a **strictly smaller** element appears

Within this range, arr[i] is guaranteed to be the **minimum**.

Algorithm

- Use a stack to find the index of the next smaller element to the right for each position
- Use another stack to find the index of the previous smaller or equal element to the left for each position

- For each element, determine how many subarrays it appears in as the minimum using its NSE(Next smaller element) and PSEE(previous smaller equal element) indices
- Calculate the contribution of each element by multiplying its value with its frequency
- Add each contribution to a total sum
- Return the total sum modulo $10^9 + 7$

Time Complexity: $O(N)$, since finding the indices of next smaller elements and previous smaller elements take $O(2*N)$ time each and calculating the sum of subarrays minimum takes $O(N)$ time.

Space Complexity: $O(N)$, since finding the indices of the next smaller elements and previous smaller elements takes $O(N)$ space each due to stack space and storing the indices of the next smaller elements and previous smaller elements takes $O(N)$ space each.

We are given an array **asteroids** of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Input: `asteroids = [3,5,-6,2,-1,4]`

Output: `[-6,2,4]`

Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Approach:

 **Core Idea**

Asteroids move in a **straight line**.

- **Positive value (> 0)** → moving **right**
- **Negative value (< 0)** → moving **left**

👉 **Collisions only happen when a right-moving asteroid meets a left-moving asteroid.**

This naturally suggests a **stack**.

📦 Why Stack?

The stack stores asteroids that are **still alive** and could collide with future asteroids.

- Top of stack = **nearest previous asteroid**
- Only the **latest right-moving asteroid** can collide with a new left-moving one

Algorithm:

1. Create an empty stack

👉 *Reason:* A stack helps us keep track of the **last asteroid** that can collide with the current one.

2. Traverse the asteroid array from left to right

👉 *Reason:* Asteroids move in sequence; collisions depend only on previous asteroids.

3. If the current asteroid is positive, push it into the stack

👉 *Reason:* Positive asteroids move right and **cannot collide yet**—we must wait for a future left-moving asteroid.

4. If the current asteroid is negative:

- While the stack is not empty and the top is positive:
 - If $\text{top} < |\text{current}|$ → pop the top
 - 👉 *Reason:* Smaller asteroid explodes.
 - If $\text{top} > |\text{current}|$ → destroy current asteroid and stop
 - 👉 *Reason:* Bigger asteroid survives.
 - If equal → pop top and destroy current, then stop
 - 👉 *Reason:* Equal sizes destroy each other.
- If the current asteroid survives all collisions, push it into the stack
 - 👉 *Reason:* No obstacle left to destroy it.

5. Convert the stack into an array and return

👉 *Reason:* The stack now contains only surviving asteroids in correct order.

Complexity

- **Time Complexity:** $O(n)$
👉 Each asteroid is pushed and popped **at most once**.
 - **Space Complexity:** $O(n)$
👉 Stack stores surviving asteroids.
-

There are n cars on an infinitely long road. The cars are numbered from 0 to $n - 1$ from left to right and each car is present at a unique point.

You are given a 0-indexed string `directions` of length n . `directions[i]` can be either 'L', 'R', or 'S' denoting whether the i^{th} car is moving towards the left, towards the right, or staying at its current point respectively. Each moving car has the same speed.

The number of collisions can be calculated as follows:

- When two cars moving in opposite directions collide with each other, the number of collisions increases by 2.
- When a moving car collides with a stationary car, the number of collisions increases by 1.

After a collision, the cars involved can no longer move and will stay at the point where they collided. Other than that, cars cannot change their state or direction of motion.

Return the total number of collisions that will happen on the road.

Input: directions = "RLRSLL"

Output: 5

Explanation:

The collisions that will happen on the road are:

- Cars 0 and 1 will collide with each other. Since they are moving in opposite directions, the number of collisions becomes $0 + 2 = 2$.
- Cars 2 and 3 will collide with each other. Since car 3 is stationary, the number of collisions becomes $2 + 1 = 3$.
- Cars 3 and 4 will collide with each other. Since car 3 is stationary, the number of collisions becomes $3 + 1 = 4$.
- Cars 4 and 5 will collide with each other. After car 4 collides with car 3, it will stay at the point of collision and get hit by car 5. The number of collisions becomes $4 + 1 = 5$.

Thus, the total number of collisions that will happen on the road is 5.

Core Insight (Very Important)

Only **right-moving cars (R)** can cause future collisions.

Why?

- Left-moving cars (L) only collide with things **to their left**
- Stationary cars (S) only affect cars **coming from the left**
- Cars to the right are irrelevant

So, we only need to **remember pending Rs**.

BETTER APPROACH (Stack-Based Simulation)

◆ Thought Process

1. We process cars **from left to right**.
2. We keep a stack representing the **current road state to the left**.
3. The stack stores cars whose collisions are **not yet resolved**.
4. Whenever a collision is possible, we **resolve it immediately**.

◆ Why Stack?

- Collisions happen with **nearest unresolved cars**
 - Stack naturally gives access to the **last car on the left**
 - Helps simulate **chain collisions**
-

◆ How collisions are handled (Conceptually)

Case 1: Current car is R

- It may collide in the future
 - So we **store it**
 - No collision now
-

Case 2: Current car is L

- If there is a right-moving car before it:
 - Head-on collision happens
 - Both cars stop → collision count increases
 - Remaining right-moving cars will hit the stopped cars
 - Final result is **one stationary car**
 - If there is already a stationary car:
 - Rear-end collision happens
-

Case 3: Current car is S

- All pending right-moving cars crash into it
 - Each causes one collision
 - The result is again **one stationary car**
-

◆ Key Principle

The stack always represents a **collision-free state** after processing each car.

◆ Why This Works

- Every collision is resolved as soon as it becomes possible
- Chain reactions are handled correctly
- The stack implicitly remembers:
 - pending right-moving cars
 - stationary barriers

Time: $O(n)$

Space: $O(n)$

🟢 OPTIMAL APPROACH (Without Stack — Counting Logic)

◆ Key Question (The Turning Point)

Do we really care **where** right-moving cars are?

Answer: ❌ No

We only care **how many** there are.

Why?

- All right-moving cars behave the same
- Once a collision starts, **every pending R will eventually collide**
- Their positions never change the final collision count

◆ Thought Process

1. Replace the stack of Rs with a **simple counter**
2. Replace stationary cars (S) with a **boolean flag**
3. Track only the **minimum necessary information**

◆ What We Track

- Number of unresolved right-moving cars
- Whether a stationary barrier exists

This fully represents the road state.

◆ Collision Logic (Conceptual)

When we see R

- Increase the count of pending right-moving cars
 - No collision yet
-

When we see S

- All pending right-moving cars crash into it
 - Collision count increases by number of pending Rs
 - Road becomes stationary
-

When we see L

- If there are pending right-moving cars:
 - One head-on collision occurs
 - Remaining right-moving cars hit the stationary cars
- If the road is already stationary:
 - Rear-end collision occurs

Time: $O(n)$

Space: $O(1)$

You are given an integer array `nums`. The range of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return the sum of all subarray ranges of `nums`.

A subarray is a contiguous non-empty sequence of elements within an array.

Input: `nums = [1,2,3]`

Output: 4

Explanation: The 6 subarrays of `nums` are the following:

`[1]`, range = largest - smallest = $1 - 1 = 0$

`[2]`, range = $2 - 2 = 0$

`[3]`, range = $3 - 3 = 0$

`[1,2]`, range = $2 - 1 = 1$

`[2,3]`, range = $3 - 2 = 1$

`[1,2,3]`, range = $3 - 1 = 2$

So the sum of all ranges is $0 + 0 + 0 + 1 + 1 + 2 = 4$.

Approach:

Thought Process & Approach (Theoretical)

Problem Insight

We are asked to compute:

Sum of (maximum – minimum) over all subarrays

Instead of calculating this value **for each subarray individually** (which would be too slow), we **invert the thinking**:

Key Idea (Contribution Technique)

Rather than iterating over subarrays, we ask:

How many subarrays consider `arr[i]` as the minimum?

How many subarrays consider `arr[i]` as the maximum?

If we know this:

- Each element's **contribution as a maximum**
- Each element's **contribution as a minimum**

Then:

$$\text{Answer} = \sum(\text{contribution as max}) - \sum(\text{contribution as min})$$

This reduces the problem to **counting influence ranges** for each element.

❄ Step 1: When is an element the MINIMUM of a subarray?

For an element $arr[i]$ to be the **minimum** of a subarray:

- All elements inside that subarray must be **greater than or equal to $arr[i]$**
- The subarray must stop **as soon as a smaller element appears**

So we search for:

- **Previous smaller or equal element** on the left
- **Next strictly smaller element** on the right

These two elements define the **maximum stretch** where $arr[i]$ remains the minimum.

🔍 Why different comparisons ($>$ vs $>=$)?

Duplicates cause double counting if handled incorrectly.

To ensure:

- Every subarray is counted **exactly once**
- No overlap between equal elements

We use:

- **Left boundary:** previous **smaller or equal**
- **Right boundary:** next **strictly smaller**

This asymmetry avoids duplication.

📏 Counting Subarrays where $arr[i]$ is MIN

Once boundaries are known:

- Left choices = distance from i to previous boundary
- Right choices = distance from i to next boundary

Total subarrays where $arr[i]$ is min = left choices $\times$ right choices

Contribution of $arr[i]$ as minimum:

$$arr[i] \times (\text{left} \times \text{right})$$

✖ Step 2: When is an element the MAXIMUM of a subarray?

The logic is **symmetric** to minimum:

For $arr[i]$ to be the maximum:

- All elements must be **less than or equal to $arr[i]$**
- Stop when a **larger element appears**

So we find:

- **Previous larger or equal element** on the left
- **Next strictly larger element** on the right

Again, asymmetry ensures no duplicate counting.

🔧 Counting Subarrays where $arr[i]$ is MAX

Total subarrays where $arr[i]$ is max = left choices $\times$ right choices

Contribution of $arr[i]$ as maximum:

$$arr[i] \times (\text{left} \times \text{right})$$

🧠 Why Monotonic Stack?

We need **nearest smaller/larger elements** efficiently.

A monotonic stack:

- Maintains elements in increasing or decreasing order
- Allows finding nearest valid boundary in **$O(1)$ amortized time**

Thus:

- Entire process runs in **$O(n)$** time
 - Uses **$O(n)$** extra space
-

← Final Composition

1. Calculate total contribution as **maximum**
2. Calculate total contribution as **minimum**
3. Subtract:

$$\text{Sum of Subarray Ranges} = \sum(\text{max contributions}) - \sum(\text{min contributions})$$

Time: $O(n)$

Space: $O(n)$

Given string `num` representing a non-negative integer `num`, and an integer `k`, return *the smallest possible integer after removing `k` digits from `num`*.

Input: `num = "10200", k = 1`

Output: `"200"`

Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.

Approach:

🧩 Key Greedy Insight

To minimize a number:

- Smaller digits should appear as early as possible
- Large digits **before** small digits are harmful

👉 If a **larger digit appears before a smaller digit**, removing the larger digit gives a smaller number.

This observation leads to a **local greedy decision** that is always globally optimal.

🧠 Core Idea: Monotonic Stack

We maintain a stack such that:

- Digits in the stack are in **non-decreasing order**
- The stack represents the **best possible prefix** formed so far

How the Stack Strategy Works (Thought Process)

We iterate through digits from left to right:

For each digit:

1. Compare it with the **top of the stack**
2. If:
 - Stack top is **larger than current digit**
 - We still have deletions left ($\text{count} < k$)
3. Then:
 - Removing the larger digit will create a **smaller number**
 - So we **pop** the stack (greedy removal)

This ensures:

Bigger digits don't block smaller digits from coming earlier.

Why This Greedy Choice is Safe

- Earlier digits have **higher positional weight**
- Replacing a larger digit with a smaller one **always reduces** the number
- Once a digit is pushed, it is only removed if a **better option arrives**

This makes the algorithm **locally optimal and globally correct**.

What if we still haven't removed k digits?

If after processing all digits:

- We still need to remove digits

Then the best choice is:

Remove digits from the **end**

Why?

- Trailing digits have **least impact** on the number
- Removing from the front would increase the value

Building the Result

The stack now contains:

- Digits of the smallest possible number
- But in **reverse order** (because of stack behavior)

So we:

1. Pop all elements
2. Reverse the order to restore original sequence

Handling Leading Zeros

After removals:

- Leading zeros don't change the numeric value
- But "000123" should be "123"

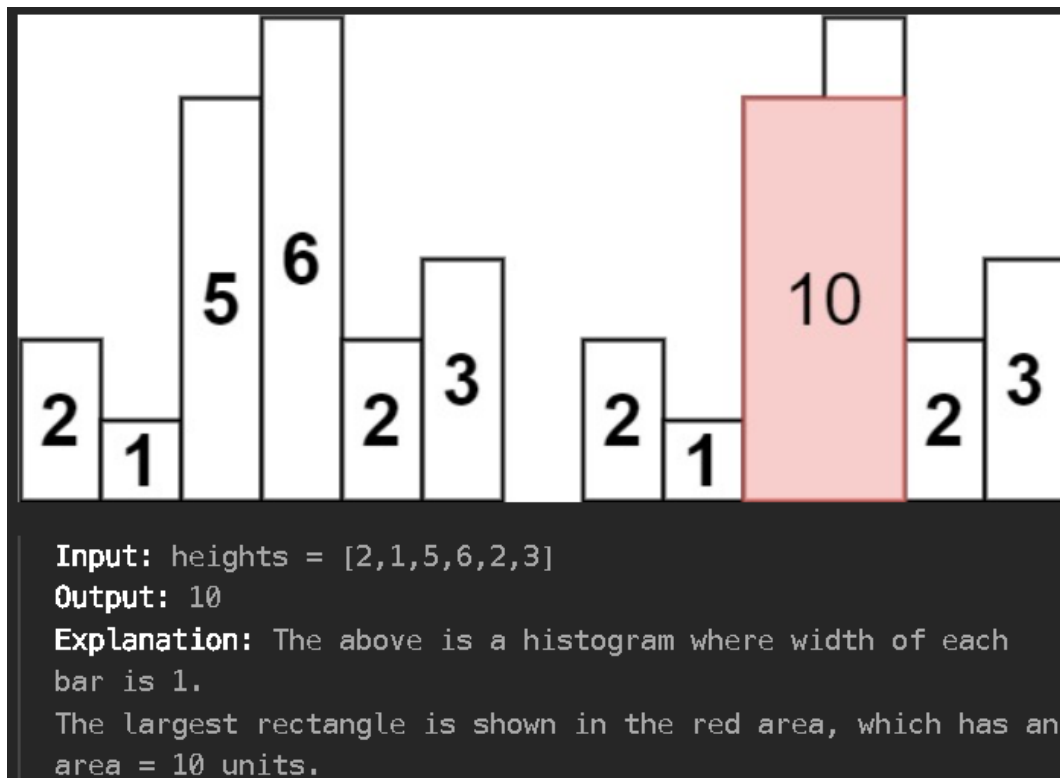
So:

- Remove extra zeros
- If everything is removed → return "0"

Time: $O(n)$

Space: $O(n)$

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram.*



Problem Intuition (Base Idea)

You are given bar heights of a histogram.

For **each bar**, imagine:

“What is the **maximum width** this bar can extend left and right **while staying the smallest bar** in that range?”

Then:

Area = height × width

The **maximum area among all bars** is the answer.

So the real question becomes:

For every index i , how far can $arr[i]$ stretch left and right before hitting a **smaller element**?

Brute Force Approach

Thought Process

For every bar:

- Expand **left** until you find a bar smaller than current

- Expand **right** until you find a bar smaller than current
- Calculate width and area

This directly simulates the definition of the rectangle.

How it works

For each index i :

1. Move left ($j = i-1$) until $\text{arr}[j] < \text{arr}[i]$
 2. Move right ($j = i+1$) until $\text{arr}[j] < \text{arr}[i]$
 3. $\text{Width} = \text{rightBoundary} - \text{leftBoundary} - 1$
 4. $\text{Area} = \text{arr}[i] \times \text{width}$
 5. Track maximum area
-

Complexity

- For each bar $\rightarrow$ scan left + right $\rightarrow O(n)$
 - For all bars $\rightarrow O(n^2)$
-

1 optimal1 — Precompute Both Boundaries (Optimized but Explicit)

Thought Process

You are thinking:

“Let me **explicitly know** the exact range where each bar can expand, and then compute the area cleanly.”

So the idea is:

- Find **Next Smaller Element (right boundary)**
 - Find **Previous Smaller Element (left boundary)**
 - Then compute area for each bar
-

Conceptual Steps

1. Right boundary (Next Smaller Element)

- For each bar, find the first bar to the right that is **strictly smaller**
- If none exists → assume boundary at the end of array

2. Left boundary (Previous Smaller Element)

- For each bar, find the first bar to the left that is **strictly smaller**
- If none exists → assume boundary before index 0

3. Area Calculation

- Now every bar knows its maximum possible width
 - Width is fixed and independent
 - Area is computed for each bar separately
 - Maximum area is chosen
-

Why this is “optimal1”

- Still $O(n)$ time
- Clear logic
- Easy to debug and understand
- Slightly more memory (two extra arrays)

This version is **optimal in time**, but not minimal in space.

Time Complexity: $O(3N)$.

Space Complexity: $O(3N)$ for the stack, left small array and a right small array

2 optimal2 — True Optimal (Single Stack, On-the-fly Area)

Thought Process

Here comes the key realization:

“Do I really need to store left and right boundaries for all bars?”

Answer: **No**.

Instead:

- The **moment a bar loses its right boundary**, its maximum area is decided.
-

Core Insight

A bar's rectangle **ends immediately** when:

- A smaller bar appears on the right

At that exact moment:

- Right boundary = current index
- Left boundary = new top of stack
- Height = popped bar

So, **area must be calculated right then.**

Conceptual Flow

1. Maintain a stack of indices with **increasing heights**
2. When a smaller bar appears:
 - Taller bars can no longer extend
 - Pop them one by one
 - For each popped bar:
 - Right boundary is known
 - Left boundary is known
 - Area is computed immediately
3. After traversal ends:
 - Remaining bars extend till the end of array
 - Their areas are calculated with right boundary as array length

Time Complexity: $O(N) + O(N)$. For loop used along with a while loop

Space Complexity: $O(N)$. Used for stack

You are given a 2D binary matrix `mat[ ][ ]`, where each cell contains either 0 or 1. Your task is to find the maximum area of a rectangle that can be formed using only 1's within the matrix.

Input: `mat[][] = [[0, 1, 1, 0],
[1, 1, 1, 1],
[1, 1, 1, 1],
[1, 1, 0, 0]]`

Output: 8

Explanation: The largest rectangle with only 1's is from (1, 0) to (2, 3) which is
[1, 1, 1, 1]
[1, 1, 1, 1]
and area is $4 * 2 = 8$.

Algorithm

- Convert the binary matrix into a matrix of histogram heights using the prefix sum technique
- For each cell in the histogram matrix, calculate the height of consecutive 1's ending at that cell
- If a cell contains a 0, reset the height to 0
- For each row in the histogram matrix, treat it as a histogram and compute the largest rectangle area
- Use a stack-based approach to find the largest rectangle in the histogram for each row
- Track the maximum rectangle area found across all rows
- Return the maximum rectangle area as the result

Time Complexity: $O(N*M)$, since filling the prefix sum matrix takes $O(N*M)$ time, and every row (of length M) is treated as a histogram for which the largest histogram is found in linear($O(2*M)$). Thus, time taking overall is $O(N*M)$.

Space Complexity: $O(N*M)$, since the prefix sum array takes up $O(N*M)$ space, and finding the largest rectangle in each histogram (of length M) takes $O(M)$ space due to stack.

Implementation Problems

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return *the max sliding window*.

Input: `nums = [1,3,-1,-3,5,3,6,7], k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

Window position	Max
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

Brute-force:

Every time the sliding window moves, we manually check all the elements within that window and pick the maximum.

- Loop through the array from index 0 to (length - window size)
- For each iteration, extract the subarray of window size
- Scan through that subarray to find the maximum element
- Store the maximum in the result array
- After the loop ends, return the result array

Time Complexity: $O(n * k)$ Each of the $(n - k + 1)$ windows is scanned completely to find its maximum. In worst-case, each window of size `k` requires $O(k)$ operations.

Space Complexity: $O(1)$ We are only using output list which does not count as extra space in space complexity analysis. No additional data structures used.

Optimal:

For each window, **only some elements are capable of becoming the maximum**.

If a new element enters the window that is **greater than some existing elements**, then those smaller elements will **never become the maximum again**, neither in the current window nor in any future window that includes the new element.

Hence, such elements are **useless** and can be discarded early.

Data Structure Choice

To efficiently:

- Remove elements from both ends
- Keep track of possible maximum candidates
- Access the maximum in constant time

we use a **Deque (Double Ended Queue)**.

The deque stores **indices**, not values, to:

1. Check whether an element is still inside the current window
 2. Retrieve values when needed
-

Invariant Maintained by the Deque

At all times, the deque satisfies **two important properties**:

1. **Window Validity**
All indices stored in the deque belong to the current window.
2. **Monotonic Decreasing Order**
The values corresponding to indices in the deque are stored in **decreasing order**.

Because of this ordering:

- The **maximum element of the window is always accessible in $O(1)$ time**.
 - Elements that can never become maximum are removed immediately.
-

Sliding the Window (Conceptual Steps)

For each new element entering the window:

1. Remove Outdated Elements

Any index that lies outside the current window range is removed, since it can no longer contribute to future results.

2. Remove Smaller Elements

If the current element is greater than or equal to elements already in the deque, those smaller elements are discarded because they will never be the maximum again.

3. Insert the Current Element

The index of the current element is added to the deque, preserving the decreasing order.

4. Record the Maximum

Once the window size reaches k , the maximum for that window is directly obtained from the deque and stored in the result.

Why This Approach Is Efficient

- Each element is **inserted once** into the deque.
- Each element is **removed at most once** from the deque.
- No element is processed repeatedly.

Thus, the total number of operations is proportional to the array size.

Complexity Analysis

- **Time Complexity:**
 $O(n)$ — each element is handled a constant number of times.
- **Space Complexity:**
 $O(k)$ — the deque stores at most k indices.

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.

The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is [7,3,4,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- StockSpanner() Initializes the object of the class.
- int next(int price) Returns the span of the stock's price given that today's price is price.

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4
prices (including today's price of 75) were less than or
equal to today's price.
stockSpanner.next(85); // return 6
```

Problem Insight (What are we trying to find?)

For each day's stock price, we need to compute the **stock span**:

The number of **consecutive days (including today)** for which the stock price was **less than or equal to today's price**, going backwards.

So for today:

- We look left
 - Stop when we find a **strictly greater** price
 - The span is the distance between today and that day
-

Naive Thought Process (Why it's inefficient)

A straightforward idea is:

- For each day, scan backward day-by-day
- Count until a higher price is found

This leads to **repeated comparisons**, making the solution $O(n^2)$ in the worst case (e.g., increasing prices).

We need a way to **reuse previous work**.

Key Observation (Optimization Idea)

When today's price is **greater than or equal to previous prices**:

- Those previous prices will **never block any future span**
- Once dominated, they become **useless**

 This leads to the idea of **removing smaller elements permanently**.

Data Structure Choice: Monotonic Stack

We maintain a **monotonic decreasing stack**:

- Stack stores **only prices greater than future prices**
- Each entry remembers:
 - the stock price
 - the day index it occurred

Stack Property

From bottom → top:

prices are strictly decreasing

Thought Process for Each New Price

When a new price arrives:

1 Remove Irrelevant Prices

- Any previous price $\leq$ current price is **popped**
- Reason: it cannot affect the span of today or any future day

This ensures the stack remains decreasing.

2 Identify the Nearest Greater Element

After popping:

- If the stack is **empty**
 - no greater price exists on the left
 - span includes **all previous days**
 - If the stack is **not empty**
 - the top element is the **nearest greater price on the left**
-

3 Compute Span Using Indices

- Span = distance between today and the nearest greater element
- If no greater element exists, treat its index as -1

This avoids scanning backward manually.

Push Current Price

- The current price becomes a candidate for future spans
 - It is pushed along with its index
-

Why This Is Efficient

- Each price is **pushed once**
- Each price is **popped once**

- No element is revisited unnecessarily

→ This gives **amortized $O(1)$ time per operation**

Complexity Analysis

Time: $O(2n)$ “Each element is pushed and popped at most once, resulting in a total of $2n$ stack operations. Hence, the overall time complexity is $O(n)$.”

Space: $O(n)$

A celebrity is a person who is known to all but does not know anyone at a party. A party is being organized by some people. A square matrix `mat[][]` of size $n \times n$ is used to represent people at the party such that if an element of row i and column j is set to 1 it means i th person knows j th person. You need to return the index of the celebrity in the party, if the celebrity does not exist, return -1.

Note: Follow 0-based indexing.

```
Input: mat[][] = [[1, 1, 0],
                  [0, 1, 0],
                  [0, 1, 1]]
```

Output: 1

Explanation: 0th and 2nd person both know 1st person and 1st person does not know anyone. Therefore, 1 is the celebrity person.

Celebrity Problem — Theoretical Approach

Problem Definition (Conceptual)

A **celebrity** is a person who:

1. **Knows nobody**
2. **Is known by everyone else**

The relationships are given in an $n \times n$ matrix:

- $mat[i][j] = 1 \rightarrow$ person i knows person j
 - $mat[i][j] = 0 \rightarrow$ person i does not know person j
-

Brute Force Approach

💡 Thought Process

The most direct way to identify a celebrity is to **explicitly verify both conditions** for every person.

For each person:

- Count how many people they **know**
- Count how many people **know them**

If a person:

- knows **0** people, and
- is known by **n-1** people,

then that person must be the celebrity.

🔍 Strategy

1. Traverse the entire matrix.
 2. For every $\text{mat}[i][j] = 1$:
 - Increase the count of people i knows
 - Increase the count of people who know j
 3. After processing all relationships:
 - Check each person
 - The one satisfying both celebrity conditions is the answer
-

❌ Why This Is Inefficient

- Every pair (i, j) is examined
 - Many unnecessary comparisons are made even after a person is clearly disqualified
-

🕒 Complexity

Metric	Value
Time	$O(n^2)$

Metric	Value
Space	$O(n)$

Optimal Approach (Elimination-Based)

Key Insight

If person **A** knows **B**, then **A** cannot be a celebrity

If person **A** does not know **B**, then **B** cannot be a celebrity

This allows us to **eliminate candidates without full verification**.

Thought Process

Instead of checking everyone fully:

- Compare people **pairwise**
- Eliminate at least **one candidate per comparison**

This guarantees we narrow down to **at most one potential celebrity**.

Candidate Elimination Phase

- Start with two pointers:
 - One at the beginning
 - One at the end
- Compare the two:
 - If top knows bottom → top is eliminated
 - If top does not know bottom → bottom is eliminated
 - If both does not know each other than both will be eliminated.
- Continue until only **one candidate remains**

At the end of this phase:

- We have **only one possible celebrity**
-

Verification Phase (Why It's Needed)

Elimination only guarantees a **possible** celebrity.

We must verify:

- The candidate knows **no one**
- Everyone else knows the candidate

This final check ensures correctness.

Complexity

Metric	Value
Elimination	$O(n)$
Verification	$O(n)$
Total Time	$O(n)$
Space	$O(1)$

Design a data structure that follows the constraints of a [Least Recently Used \(LRU\) cache](#).

Implement the LRUCache class:

- **LRUCache(int capacity)** Initialize the LRU cache with positive size capacity.
- **int get(int key)** Return the value of the key if the key exists, otherwise return -1.
- **void put(int key, int value)** Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions get and put must each run in $O(1)$ average time complexity.

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get",  
"get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3],  
[4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);  
lRUCache.put(1, 1); // cache is {1=1}  
lRUCache.put(2, 2); // cache is {1=1, 2=2}  
lRUCache.get(1);    // return 1  
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is  
{1=1, 3=3}  
lRUCache.get(2);    // returns -1 (not found)  
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is  
{4=4, 3=3}  
lRUCache.get(1);    // return -1 (not found)  
lRUCache.get(3);    // return 3  
lRUCache.get(4);    // return 4
```

◆ Problem Understanding (What is LRU?)

LRU (Least Recently Used) Cache evicts the item that was **used least recently** when capacity is exceeded.

- Every **get(key)** or **put(key, value)** makes that key **most recently used**
- When cache is full → remove the **least recently used** item
- Both operations must run in **O(1) average time**

◆ Key Challenge

We need to support **two things in O(1)**:

1. **Fast lookup by key**
2. **Fast update of usage order**

A single data structure **cannot efficiently do both** → so we combine **two**.

◆ Chosen Data Structures (Why This Works)

1 HashMap (Key → Node)

- Gives **O(1)** access to cache entries
- Maps a key to its corresponding node in the list

👉 Solves: *“How do I find an item instantly?”*

2 Doubly Linked List (Usage Order)

- Maintains order of usage
- **Most Recently Used (MRU)** → near **head**
- **Least Recently Used (LRU)** → near **tail**

👉 Solves: *“Which item should be removed?”*

🔥 Why Doubly Linked List (not singly)?

- Allows **O(1)** deletion of any node
 - We already have the node reference from the HashMap
 - No traversal needed
-

◆ Design Strategy

We use **dummy head and tail nodes** to simplify edge cases.

HEAD <-> MRU ... LRU <-> TAIL

- head.next → most recently used
 - tail.prev → least recently used
-

◆ Thought Process for Each Operation

✓ get(key)

Goal:

- Return value if key exists
- Mark the key as **most recently used**

Steps:

1. Check key in HashMap
 - If not found → return -1
2. Retrieve the node in **O(1)**
3. Remove node from its current position
4. Move node to **front (right after head)**

📌 Why move to front?

Because accessing it means it is **recently used**

🕒 Time Complexity: **O(1)**

✓ put(key, value)

Case 1: Key already exists

Goal: Update value + refresh usage

1. Fetch node using HashMap
2. Update value
3. Remove node from its current position
4. Move it to front (MRU)

🕒 Time Complexity: **O(1)**

Case 2: Key does NOT exist

Goal: Insert new key-value pair

1. If cache is **at capacity**:
 - Identify LRU → tail.prev
 - Remove it from:

- Doubly linked list
 - HashMap
2. Create a new node
 3. Insert node at front (MRU)
 4. Add entry to HashMap

📌 Eviction is $O(1)$ because:

- LRU is always at `tail.prev`
- No traversal needed

🕒 Time Complexity: **$O(1)$**

♦ Why This Achieves $O(1)$?

Operation HashMap Doubly Linked List

Lookup	$O(1)$	—
Insert	$O(1)$	$O(1)$
Delete	$O(1)$	$O(1)$
Move node	—	$O(1)$

👉 Combined operations still remain **$O(1)$**

♦ Invariants (Rules Always Maintained)

- HashMap size $\leq$ capacity
 - Every key in HashMap has a corresponding DLL node
 - Most recently used $\rightarrow$ right after head
 - Least recently used $\rightarrow$ right before tail
-

♦ Final Summary (Interview-Ready)

- **HashMap** gives fast access by key
- **Doubly Linked List** maintains LRU order

- `get()` and `put()` both:
 - Access node in $O(1)$
 - Reorder list in $O(1)$
- Eviction is constant time due to tail pointer

✓ Hence, the design satisfies **$O(1)$ average time complexity**

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- **`LFUCache(int capacity)` Initializes the object with the capacity of the data structure.**
- **`int get(int key)` Gets the value of the key if the key exists in the cache. Otherwise, returns -1.**
- **`void put(int key, int value)` Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.**

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the `put` operation). The use counter for a key in the cache is incremented either a `get` or `put` operation is called on it.

The functions `get` and `put` must each run in $O(1)$ average time complexity.

Approach: LFU Cache (Least Frequently Used)

Problem Understanding

An **LFU Cache** removes the element that has been **used the least number of times** when the cache reaches its capacity.

If multiple elements have the same frequency, the **Least Recently Used (LRU)** among them is evicted.

So, we must support:

- `get(key)` → return value & increase frequency

- `put(key, value)` → insert/update & evict if needed
👉 **All in $O(1)$ time**
-

Key Challenges

1. Tracking **frequency** of each key
2. Finding the **minimum frequency** quickly
3. Resolving **ties using LRU**
4. Ensuring **$O(1)$** time complexity

A single data structure cannot handle all of this efficiently, so we **combine multiple structures**.

Data Structures Used & Why

1 HashMap (key → node)

- Direct access to cache entries
- Enables **$O(1)$** lookup for get and put

Each node stores:

- key
 - value
 - frequency
 - pointers for DLL navigation
-

2 HashMap (frequency → Doubly Linked List)

- Groups nodes by their frequency
- Each frequency has its **own DLL**
- DLL maintains **LRU order within same frequency**

Why DLL?

- $O(1)$ insertion
- $O(1)$ deletion

- Required for LRU handling
-

3 Minimum Frequency Variable

- Keeps track of the **lowest frequency currently present**
 - Allows instant access to eviction candidate
-

Working Logic

◆ Get Operation

1. Check if key exists
 - If not → return -1
2. If exists:
 - Remove node from its current frequency list
 - Increase its frequency by 1
 - Add it to the new frequency list
3. If the old frequency list becomes empty:
 - Update minFrequency
4. Return the value

👉 This ensures **usage frequency is always updated correctly**

◆ Put Operation

Case 1: Cache capacity is zero

- Do nothing

Case 2: Key already exists

- Update its value
- Increase its frequency (same steps as get)

Case 3: New key insertion

1. If cache is full:
 - Locate list with minFrequency

- Evict the **LRU node** from that list
 - 2. Insert new node with:
 - frequency = 1
 - 3. Update:
 - minFrequency = 1
 - current size
-

Eviction Policy (Core Idea)

1. **Least Frequency First**
2. If tie → **Least Recently Used**

This is achieved by:

- Frequency-based grouping
 - DLL-based ordering within each frequency
-

Why This Design Works (Thought Process)

- **HashMap** gives fast access by key
- **Frequency Map** organizes nodes by usage
- **DLL** resolves LRU efficiently
- **minFrequency** avoids scanning frequencies

All operations touch **constant number of nodes**, so:

- `get()` → $O(1)$
 - `put()` → $O(1)$
-

Time & Space Complexity

Time Complexity

Operation	Complexity
get	$O(1)$

Operation	Complexity
-----------	------------

put	$O(1)$
-----	--------

Space Complexity

- $O(\text{capacity})$
(Each element stored once across maps & DLLs)